

Data Flywheel for Autonomous Systems

Design for off-road perception at fleet scale

Nakshatra Vyas — Data Engineer, Autonomous Systems

Zensomy Autonomous Technologies · October 2026

Ingestion · selection · training · release · the three edges that close the loop



A pipeline runs and stops. A flywheel keeps momentum

PIPELINE

New data arrives. The same steps run again. Each run costs what the last one cost and buys what the last one bought.

FLYWHEEL

Each turn makes the next turn cheaper, because information also flows backwards.

The feedback edges are the design. Take them out and this is a pipeline that is run often.

Three edges, named and built: field failures fire retraining · weak evaluation slices steer the next collection · shadow disagreements become labelling candidates.



The whole loop, one picture



Getting data off the machine

1 · TELEMETRY

Always on. Tiny.

2 · CELLULAR CLIPS

Flagged events only.

3 · DEPOT WIFI

Bulk — about **98% of bytes**, near-zero cost.

4 · PHYSICAL MEDIA

Sites with no link.

INTEGRITY — HARD FAIL

Did all the bytes arrive? Hashes, chunk counts, sequence gaps.
→ **quarantine**

QUALITY — SOFT FLAG

Is the data usable? Flagged, not dropped. A dusty lens is what selection wants. → **raw**

Cost Tier 3 assumes machines come back to a depot most days — the biggest assumption here, and tier 4 is its fallback. **Calibration version is required at ingest:** a mount that shifts raises no error, it just poisons every cross-sensor projection.

Four signals, because uncertainty cannot see confident error

WRONG

Disengagement, emergency brake, near-miss. **Needs no model.**

UNSURE

Low confidence, narrow margin, unstable detection.

CONFLICT

Camera and LiDAR disagree. Gated on clock sync.

RARE

Gap against the scenario list.

A model that is certain and wrong is the worst case: the machine drives into it at full speed. Only WRONG and CONFLICT can see that clip.

WEIGHTS

WRONG counts most · then UNSURE · then CONFLICT · then RARE

Cost More scoring machinery to build and keep running than plain uncertainty sampling.

What the ranking looks like

CLIP	OUTCOME
Disengagement in dust	Must - take
LiDAR sees it, camera does not	Selected
Same dust, a minute later	Deferred — a near neighbour was already taken
Straight line, clear day	Random reserve

Quality multiplies

A faulty sensor looks maximally interesting. A subtracted penalty gets outvoted. A multiplier holds the clip back and raises a maintenance alert.

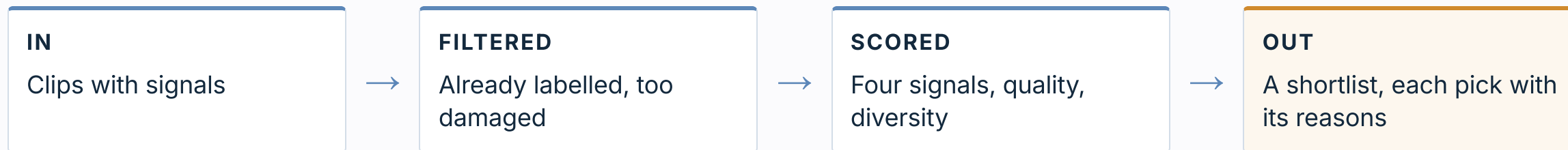
Every pick carries its reasons, so anyone can ask why we paid for a clip. The random reserve keeps evaluation honest — without it, normal-condition performance stops being measurable.

Three groups, not a top slice

Ranking the shortlist on the cheap signals admits only event clips, and excludes exactly what UNSURE and CONFLICT exist to find.



The selection component is built



- **Each test name is a promise.** One proves the four-signal argument: uncertainty alone ranks the dangerous clip **last**.
- **Nothing is dropped silently.** Every skipped clip records a reason, and a test asserts that input equals dropped plus scored.
- **A missing signal is not a zero signal.** With no model yet, UNSURE returns nothing and the rest are reweighted.

Two dependencies. `docker compose up --build`. The run is repeatable: ties break on clip ID, and the manifest hashes the input.

A dataset version is a manifest, and a run has four pins

FIXED MANIFEST

References plus content hashes, written once. A folder is editable — clips come and go and the name stays the same.

STICKY SPLITS

Set per session and never redrawn. Nearby frames are near-duplicates, so a frame split puts twins in train and test.

LEAKAGE CHECK

Compares content, not IDs. A session ingested twice is one drive with two names.

DATASET HASH

GIT SHA

IMAGE DIGEST

not a tag

CONFIG AND SEEDS

A leaky split does not look like an error. It looks like success. **Cost** Sticky splits waste some data, but redrawing them breaks every past comparison. Reproducibility is statistical, not bitwise, and we say so — claiming more than we paid for is worse than claiming none.



Gate on slices, never on the aggregate

AGGREGATE

+1.2
points overall. Looks like a clear win.

DUST SLICE

-5.7
points. For a machine in a dry field, that model is worse.

A small slice moves the headline number by almost nothing, so a bad regression inside it stays invisible — and the rare slices are the costly ones.

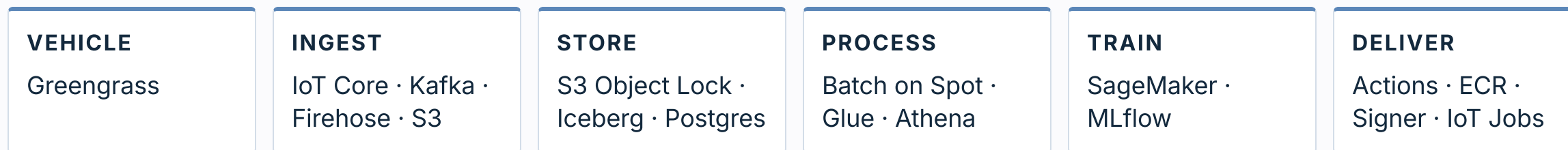
- **A frozen evaluation set**, refreshed on purpose. A growing exam means models sat different tests.
- **A regression suite of real field incidents that only grows**. Nothing is removed, because the conditions have not stopped existing.

Release a bundle, then roll it out in stages



Accuracy depends on code around the model, so the bundle is the release unit. **The artifact that trains is not the artifact that runs** — quantising hurts low-contrast scenes most, so the compiled model is retested on target hardware. On the machine, two slots and an automatic revert, which works with no network and no human. **Cost** One bundle couples cadence: a one-line fix needs a full release.

Buy managed, build only the selection logic



The scarcest resource is not compute or storage. It is engineering attention.

- **Buy managed unless managed forces a design compromise.** Kafka is the one bought exception — mine sites have no dependable link, so the streaming layer must be able to run locally.
- **Build only selection and curation.** The rest is plumbing, and plumbing is rented.

What breaks first is not the cloud

BINDS FIRST	WHY	RESPONSE
Depot upload bandwidth	A link that serves five machines does not serve fifty	Cut more on the machine; grow the physical tier
The labelling budget	Ten times the data is not ten times the budget	Selection carries more weight
Bench hardware for hardware-in-the-loop	Scales with vehicle variants , not fleet size	One bench per variant; compile and test in parallel



The three edges that close the loop

- ① **Field failures → retraining.** A deployed model's slice failure rate crosses a threshold and fires the pipeline that replaces it.
- ② **Weak slices → next collection.** A slice that scores badly gets more weight next cycle. Evaluation stops being a report and becomes an input.
- ③ **Shadow disagreements → selection pool.** A frame two models disagree on is hard by construction. [Testing a model makes the data that improves the next one.](#)

Returns build rather than repeat: a random hour is worth less each cycle, so selection must get sharper · a better model pre-labels better, so the same budget buys more labelled hours · every release leaves new regression tests behind.

Five trade-offs, with the cost stated

CHOSE	OVER	COST ACCEPTED
Depot WiFi for most bytes	One upload path	Depends on a daily depot return; physical media is the fallback
Four selection signals	Uncertainty sampling alone	More scoring machinery to build and run
Sticky session splits	Random frame split	Wastes some data; splits cannot be balanced exactly
The bundle as release unit	Separate model and software deploys	A one-line fix needs a full release
Statistical reproducibility, stated	Bitwise determinism	Re-runs vary within normal variance

The full register states each decision as **what was chosen** · **what it was chosen over** · **what it costs** — the named trade-offs matter more than the stack.

Three claims this design makes

It is a flywheel, not a pipeline. Three feedback edges are named and built. Take them out and this is a pipeline that is run often.

Every gate fails closed. No stage produces output by default. A label batch, a dataset version, a model, a release bundle — each exists because a check passed, and failed attempts are recorded as fully as successful ones.

It is sized for this company, and it says where it stops. Every place that assumption binds is named, with the trigger that changes the decision.

Thank you

Happy to go deeper on any part of it — selection, the release gate, or how the loop closes.

Nakshatra Vyas · Data Engineer — Autonomous Systems

